

Java Backend Architecture Interview Series

Sub-Chapter 1.2

Collections Framework & List

ArrayList | LinkedList | CopyOnWriteArrayList | Iteration | ConcurrentModificationException |
Immutable Lists

40+ pages & 28 Q&A | Standard + Advanced Interview Questions | Working Code Examples

1 Collections Framework Architecture

1.1 Introduction & Motivation

The Java Collections Framework (JCF) is a unified architecture for representing and manipulating groups of objects. Before it existed (pre-Java 2), each vendor and developer invented their own data structures with incompatible interfaces. The JCF standardised this into a clean interface hierarchy: code to interfaces (List, Set, Map), swap implementations without changing callers. Understanding the hierarchy is essential for choosing the right data structure and for answering the perennial interview question: 'which collection would you use and why?'

1.2 Interface Hierarchy (ASCII Diagram)

```
java.lang.Iterable<T>

+-- java.util.Collection<E>

+-- java.util.List<E> (ordered, indexed, allows duplicates)
| +-- ArrayList (resizable array)
| +-- LinkedList (doubly-linked, also Deque)
| +-- Vector (legacy, synchronized)
| +-- CopyOnWriteArrayList(thread-safe snapshot iterator)
|
+-- java.util.Set<E> (no duplicates)
| +-- HashSet (hash table, O(1) ops)
| +-- LinkedHashSet (insertion-ordered hash table)
| +-- SortedSet -> NavigableSet
| +-- TreeSet (Red-Black tree, O(log n))
|
+-- java.util.Queue<E> (FIFO semantics)
+-- PriorityQueue (heap-based)
+-- java.util.Deque<E> (double-ended queue)
+-- ArrayDeque (resizable circular array)
+-- LinkedList (also implements List!)

// Map hierarchy (SEPARATE -- does NOT extend Collection)

java.util.Map<K,V>

+-- HashMap (hash table, O(1) avg)
```

```
+-- LinkedHashMap (insertion/access-ordered)

+-- Hashtable (legacy, synchronized)

+-- SortedMap -> NavigableMap

+-- TreeMap (Red-Black tree, O(log n))
```

Collections Framework complete interface hierarchy

1.3 Key Design Principle: Code to Interfaces

```
import java.util.List;
import java.util.ArrayList;

// BAD locked to implementation
ArrayList<String> list = new ArrayList<>();

// GOOD code to interface; swap implementation freely
List<String> list = new ArrayList<>(); // or LinkedList, unmodifiableList, etc.

// Service method: accept interface, not implementation
public void process(List<String> items) { ... } // callers can pass any List
```

Code to interface principle

1.4 Factory Methods Overview (Java 9+)

```
import java.util.List;
import java.util.Set;
import java.util.Map;
import java.util.ArrayList;

// Immutable collections via factory methods (Java 9+)
List<String> names = List.of("Alice", "Bob", "Carol"); // immutable
Set<Integer> ids = Set.of(1, 2, 3); // immutable
Map<String,Integer>m = Map.of("a",1, "b",2); // immutable

// Mutable copy from immutable
List<String> mutable = new ArrayList<>(List.of("a", "b", "c"));

// Null hostile! List.of(null) throws NullPointerException
// Duplicate hostile in Set: Set.of(1,1) throws IllegalArgumentException
```

Java 9+ factory methods for immutable collections

2 ArrayList Deep Dive

2.1 Introduction The Go-To General Purpose List

ArrayList is the most commonly used List implementation and the default choice for most use cases. It is backed by a dynamically resizing array, providing $O(1)$ random access and amortized $O(1)$ appends. The key design decision: when the backing array is full, ArrayList allocates a new array at 1.5x the old size and copies all elements. Understanding this growth behaviour explains when to pre-size with `initialCapacity` and when ArrayList is NOT the right choice.

2.2 Before The Problem with Fixed Arrays

```
import java.util.Arrays;

// Before ArrayList: raw arrays require manual resize

String[] arr = new String[10];

int size = 0;

// arr[10] = "overflow"; // ArrayIndexOutOfBoundsException!

// To add more: manually copy to larger array

arr = Arrays.copyOf(arr, arr.length * 2);

// Tedious, error-prone, no type safety beyond E[]
```

Before: manual array management

2.3 After ArrayList Internals

```
import java.util.List;
import java.util.ArrayList;

// ArrayList backed by Object[] elementData

// Default initial capacity: 10 (Java 8+: lazy init, 0 until first add)

List<String> list = new ArrayList<>(); // initial capacity 0 (lazy)

List<String> sized = new ArrayList<>(1000); // pre-allocated capacity 1000

// Growth: newCapacity = oldCapacity + (oldCapacity >> 1) approx 1.5x
// At capacity 10: next resize -> 15 -> 22 -> 33 -> 49 ...

// Core operations and complexities

list.add("a"); // O(1) amortized append at end

list.add(0, "first"); // O(n) shifts all elements right

list.get(5); // O(1) direct array index

list.remove(0); // O(n) shifts all elements left

list.remove("a"); // O(n) linear scan + shift

list.contains("a"); // O(n) linear scan
```

```
list.size(); // O(1)

// Capacity management
list.ensureCapacity(500); // pre-expand to avoid multiple resizes
list.trimToSize(); // shrink backing array to current size (save memory)
```

ArrayList: core operations and complexity

2.4 When to Use ArrayList vs When to Avoid It

Use ArrayList When	Avoid ArrayList When
Mostly get(index) / set(index) operations	Frequent insertions/deletions in the middle
Iterating over elements sequentially	Need true O(1) random insertions
Single-threaded access	Multiple threads write concurrently
Memory efficiency matters (compact storage)	Many frequent add(0, x) prepend operations

2.5 Sorting and Searching

```
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.stream.Collectors;

List<String> names = new ArrayList<>(List.of("Charlie", "Alice", "Bob"));

// Sort (mutates the list)
Collections.sort(names); // natural order: [Alice, Bob, Charlie]
names.sort(Comparator.reverseOrder()); // reverse: [Charlie, Bob, Alice]
names.sort(String::compareToIgnoreCase);

// Binary search (list MUST be sorted first!)
Collections.sort(names);
int idx = Collections.binarySearch(names, "Bob"); // O(log n)

// Stream-based operations
List<String> filtered = names.stream()
    .filter(s -> s.startsWith("A"))
    .sorted()
    .collect(Collectors.toList());

// Java 16+ toList() -- returns unmodifiable list
List<String> result = names.stream().sorted().toList();
```

ArrayList: sorting, searching, and stream operations

3 LinkedList Deep Dive

3.1 Introduction Doubly-Linked List + Deque

LinkedList implements both List and Deque interfaces. Internally it is a doubly-linked list: each node holds a reference to the previous and next node. This gives $O(1)$ insertions/deletions at the head and tail, but $O(n)$ random access because the list must traverse from head (or tail) to find element at index i . Contrary to popular belief, LinkedList is rarely the better choice over ArrayList in practice due to poor cache locality and higher memory overhead per element.

3.2 LinkedList Node Structure & Memory Overhead

```
import java.util.LinkedList;
import java.util.Deque;
import java.util.List;

// Each node: ~48 bytes on HotSpot with compressed oops
// object header (16) + prev ref (4) + next ref (4) + item ref (4) + padding
// vs ArrayList element: just 4 bytes (reference in array)

LinkedList<String> list = new LinkedList<>();

// O(1) head/tail operations
list.addFirst("head"); // O(1)
list.addLast("tail"); // O(1)
list.peekFirst(); // O(1) returns null if empty
list.peekLast(); // O(1)
list.pollFirst(); // O(1) removes and returns, null if empty
list.removeFirst(); // O(1) throws NoSuchElementException if empty

// O(n) middle operations
list.get(5); // O(n) traverses to index 5
list.add(5, "mid"); // O(n) find position then O(1) link update

// As Deque (double-ended queue)
Deque<String> deque = new LinkedList<>();
deque.push("a"); // push to front O(1)
deque.pop(); // remove from front O(1)
deque.offer("b"); // add to tail O(1)
deque.poll(); // remove from head O(1)
```

LinkedList: node structure and Deque interface usage

3.3 ArrayList vs LinkedList The Real Comparison

Operation	ArrayList	LinkedList	Winner
get(index)	O(1)	O(n)	ArrayList
add(E) -- append	O(1) amortized	O(1)	Tie
add(0, E) -- prepend	O(n)	O(1)	LinkedList
add(mid, E) -- middle	O(n)	O(n)*	ArrayList (cache)
remove(0) -- head	O(n)	O(1)	LinkedList
remove(mid)	O(n)	O(n)*	ArrayList (cache)
contains()	O(n)	O(n)	Tie
Memory per element	~4 bytes ref	~48 bytes node	ArrayList
Cache efficiency	Excellent	Poor	ArrayList

* LinkedList middle insert/remove: O(n) to find the node, then O(1) to relink. But poor cache locality makes it slower than ArrayList in practice even for these ops.

TIP: In practice, ArrayList outperforms LinkedList for almost all use cases because of CPU cache locality. LinkedList's advantage is only real for frequent O(1) head/tail operations where ArrayDeque is usually a better choice anyway (ArrayDeque has better cache locality).

4 CopyOnWriteArrayList Thread-Safe List

4.1 The Problem Thread Safety Without Locks on Reads

In a multi-threaded environment, `ArrayList` is NOT thread-safe: concurrent writes cause data corruption and concurrent read+write causes `ConcurrentModificationException`. The traditional fix (`Collections.synchronizedList`) synchronises every operation, blocking all readers whenever a writer is active. `CopyOnWriteArrayList` takes a different approach: reads are fully lock-free (iterate without any synchronization), and every write creates a NEW copy of the backing array. This makes it ideal for read-heavy, write-rare scenarios.

4.2 How CopyOnWriteArrayList Works

```
import java.util.concurrent.CopyOnWriteArrayList;

// CopyOnWriteArrayList -- thread-safe, lock-free reads

CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();

// Write: acquires lock, copies array, modifies copy, publishes atomically
list.add("hello"); // O(n): copies entire array + adds element
list.remove("hello"); // O(n): copies entire array

// Read: completely lock-free -- operates on stable snapshot
String s = list.get(0); // O(1) on snapshot
list.contains("hello"); // O(n) on snapshot

// Iterator: operates on SNAPSHOT at time of iterator creation
// Modifications during iteration do NOT affect the snapshot
// remove() on iterator throws UnsupportedOperationException
for (String item : list) {
    System.out.println(item); // safe -- no ConcurrentModificationException
    // list.remove(item); // OK on the live list, but won't affect loop
}
```

CopyOnWriteArrayList: read/write semantics and snapshot iteration

4.3 When to Use CopyOnWriteArrayList

Use CopyOnWriteArrayList When	Avoid CopyOnWriteArrayList When
Reads vastly outnumber writes (event listener lists)	Frequent writes -- O(n) copy per write
Need lock-free iteration in multi-threaded code	Large lists -- copying is expensive
Observer/subscriber pattern with occasional add/remove	Memory-sensitive -- two arrays during copy
Publishing configuration snapshots	Using <code>iterator.remove()</code> -- throws UOE

WARNING: CopyOnWriteArrayList is NOT a general-purpose replacement for ArrayList in concurrent code. Its $O(n)$ write cost makes it unsuitable for high-write workloads. For high-concurrency lists, consider ConcurrentLinkedQueue or partition-based approaches.

5 Iteration Patterns & ConcurrentModificationException

5.1 Introduction Four Ways to Iterate a List

Java provides multiple iteration patterns for List, each with different semantics: indexed for-loop (allows mutation of elements but not structure), for-each (syntactic sugar for Iterator, cleaner but no remove), Iterator (allows safe remove during iteration), ListIterator (bidirectional, add/set allowed), and Stream (functional, parallel-capable). Choosing the wrong pattern is the number-one cause of ConcurrentModificationException.

5.2 All Iteration Patterns Side by Side

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.ListIterator;

List<String> list = new ArrayList<>(List.of("a","b","c","d"));

// Pattern 1: indexed for-loop O(1) get per step for ArrayList
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i)); // can also set(i, ...) during loop
}

// Pattern 2: for-each uses Iterator internally; clean syntax
for (String s : list) {
    System.out.println(s); // DO NOT call list.remove() inside!
}

// Pattern 3: Iterator safe remove() during iteration
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) it.remove(); // safe remove
}

// Pattern 4: ListIterator bidirectional + add/set
ListIterator<String> li = list.listIterator();
while (li.hasNext()) {
    String s = li.next();
    li.set(s.toUpperCase()); // replace in-place
    li.add("*"); // insert after current position
}
```

```
}

// Pattern 5: Stream functional, composable, parallel
list.stream()
    .filter(s -> !s.isEmpty())
    .map(String::toUpperCase)
    .forEach(System.out::println);

// Pattern 6: removeIf Java 8+ clean conditional remove
list.removeIf(s -> s.startsWith("a")); // removes all matching
```

Six iteration patterns for List

5.3 ConcurrentModificationException The modCount Mechanism

WARNING: NEVER modify a List structurally (add/remove) while iterating with for-each or Iterator on fail-fast collections. Use `Iterator.remove()`, `removeIf()`, or `collect-then-remove` patterns instead.

```
import java.util.List;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.stream.Collectors;

// HOW IT WORKS: ArrayList maintains an int modCount field
// Every structural modification (add/remove) increments modCount
// Iterator captures expectedModCount = modCount at creation time
// Iterator.next() checks: if (modCount != expectedModCount) throw CME

// WRONG throws ConcurrentModificationException
for (String s : list) {
    if (s.equals("b")) list.remove(s); // modCount changes -> CME!
}

// FIX 1: Iterator.remove() does NOT increment the check counter
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) it.remove(); // safe
}

// FIX 2: removeIf (Java 8+) cleanest
list.removeIf(s -> s.equals("b"));

// FIX 3: collect to remove list, then removeAll
List<String> toRemove = list.stream()
```

```
.filter(s -> s.equals("b")).collect(Collectors.toList());  
  
list.removeAll(toRemove);  
  
// FIX 4: iterate backward by index avoid shift issues  
for (int i = list.size()-1; i >= 0; i--) {  
    if (list.get(i).equals("b")) list.remove(i);  
}
```

ConcurrentModificationException: cause and all four fixes

5.4 Fail-Fast vs Fail-Safe Iterators

Property	Fail-Fast (ArrayList iterator)	Fail-Safe (CopyOnWriteArrayList iterator)
Modification check	Yes -- throws CME on structural modification	No -- iterates over snapshot
Memory	No extra memory	Snapshot copy allocated
Freshness	Always sees current state	May not see new elements
remove()	Iterator.remove() OK; list.remove() = CME	UnsupportedOperationException
Use when	Single-threaded or needs up-to-date view	Multi-threaded read iteration

6 Immutable Lists & List Performance Comparison

6.1 Introduction Three Ways to Create Immutable Lists

Java offers multiple ways to create read-only or immutable lists. Understanding the differences between them is a common interview question: `List.of()` is truly immutable (null-hostile, no modification at all), `Collections.unmodifiableList()` is a view over a mutable list (the underlying list can still change), and `Collections.singletonList()` creates a size-1 immutable list. Choosing wrongly can lead to unexpected `UnsupportedOperationException` or surprising behaviour when the underlying list changes.

6.2 Three Approaches to Immutable Lists

```
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;

// Approach 1: List.of() (Java 9+) truly immutable
List<String> immutable = List.of("a", "b", "c");

// immutable.add("d"); // UnsupportedOperationException
// immutable.set(0,"x"); // UnsupportedOperationException
// List.of(null); // NullPointerException null-hostile!
// Compact representation may use array-based specialisation

// Approach 2: Collections.unmodifiableList() VIEW (not truly immutable)
List<String> mutableBase = new ArrayList<>(List.of("a", "b", "c"));
List<String> view = Collections.unmodifiableList(mutableBase);

// view.add("d"); // UnsupportedOperationException
mutableBase.add("d"); // OK! modifies the underlying list
System.out.println(view); // [a, b, c, d] view reflects the change!

// Approach 3: Collections.singletonList() exactly one element
List<String> single = Collections.singletonList("only");
// single.add("x"); // UnsupportedOperationException

// Making a true immutable copy from existing list
List<String> copy = List.copyOf(mutableBase); // Java 10+ immutable copy
// copy does NOT reflect subsequent changes to mutableBase
```

Three immutable list approaches and their differences

6.3 Immutable List Comparison Table

Method	Truly Immutable?	Null Elements	Java Version	Notes
List.of()	Yes	Rejected (NPE)	Java 9+	Most efficient; compact internal representation
Collections.unmodifiableList()	No (view)	Allowed	Java 1.2+	Underlying list can change; callers see changes
Collections.singletonList()	Yes	Allowed (1 null)	Java 1.3+	Only 1 element; useful for single-element APIs
List.copyOf()	Yes	Rejected (NPE)	Java 10+	Makes an immutable copy of an existing collection
Arrays.asList()	No	Allowed	Java 1.2+	Fixed size (no add/remove); set() allowed

WARNING: Arrays.asList() is frequently misunderstood: it returns a FIXED-SIZE list backed by the array. add() and remove() throw UnsupportedOperationException, but set() is allowed. It also reflects changes to the original array.

6.4 List Performance Comparison Summary

Operation	ArrayList	LinkedList	CopyOnWriteArrayList
get(index)	O(1)	O(n)	O(1)
add(E) -- append	O(1) amort	O(1)	O(n) -- copies array
add(i, E) -- mid	O(n)	O(n)	O(n)
remove(index)	O(n)	O(n)	O(n)
contains()	O(n)	O(n)	O(n)
Iterator	Fail-fast	Fail-fast	Fail-safe (snapshot)
Thread-safe?	No	No	Yes (reads lock-free)
Memory per elem	~4 bytes	~48 bytes	~4 bytes + copy on write

7 Advanced List Patterns

7.1 subList, Bulk Operations & Collections Utilities

```
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;

List<Integer> nums = new ArrayList<>(List.of(0,1,2,3,4,5,6,7,8,9));

// subList -- returns a VIEW (not a copy)
List<Integer> sub = nums.subList(2, 6); // [2,3,4,5]
sub.set(0, 99); // nums is now [0,1,99,3,4,5,6,7,8,9]
sub.clear(); // removes elements 2-5 from nums

// Common pattern: delete range from list
nums.subList(2, 5).clear(); // efficient bulk delete

// Collections utility methods
Collections.shuffle(nums); // random shuffle
Collections.reverse(nums); // reverse in-place
Collections.fill(nums, 0); // fill with value
Collections.nCopies(5, "x"); // unmodifiable [x,x,x,x,x]
Collections.frequency(nums, 0); // count occurrences
Collections.max(nums); // max by natural order
Collections.min(nums, Comparator.reverseOrder()); // min by custom order
Collections.disjoint(list1, list2); // true if no common elements
```

subList (view semantics) and Collections utility methods

WARNING: `subList()` returns a LIVE VIEW. Structural modifications to the original list while holding a `subList` reference throws `ConcurrentModificationException`. To get an independent copy: `new ArrayList<>(list.subList(from, to))`.

7.2 Stream Collectors for Lists

```
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

List<String> words = List.of("banana","apple","cherry","apricot");

// Collect to mutable ArrayList
List<String> mutable = words.stream()
```

```
.filter(w -> w.startsWith("a"))  
  
.collect(Collectors.toList()); // mutable, may be ArrayList  
  
// Collect to unmodifiable list  
List<String> unmod = words.stream()  
    .sorted()  
    .collect(Collectors.toUnmodifiableList()); // Java 10+  
  
// Java 16+ toList() unmodifiable, more concise  
List<String> result = words.stream().sorted().toList();  
  
// Grouping into Map<key, List<value>>  
Map<Character, List<String>> byLetter = words.stream()  
    .collect(Collectors.groupingBy(w -> w.charAt(0)));  
  
// {a=[apple,apricot], b=[banana], c=[cherry]}
```

Stream collectors for List creation and grouping

Q&A Interview Questions & Answers 28 Questions

Standard questions are mid-level. [ADV] questions target senior/principal interviews.

Part 1 Collections Framework Architecture

Q1. Why does Map NOT extend Collection?

A Map is not a collection of elements -- it is a collection of key-value mappings. Mixing the two would break the semantics of methods like `add(E e)`, `contains(Object o)`, etc. For example, what would `add()` mean for a Map entry? The design keeps Collection for single-element containers and Map separate.

Q2. What is the difference between Collection and Collections?

Collection (`java.util.Collection`) is an interface -- the root interface of the Collection hierarchy (List, Set, Queue extend it). Collections (`java.util.Collections`) is a final utility class with static methods: `sort()`, `shuffle()`, `unmodifiableList()`, `synchronizedList()`, `min()`, `max()`, etc.

Q3. What does coding to an interface mean and why does it matter?

Declaring variables as the interface type: `List list = new ArrayList<>()` not `ArrayList list`. Benefits: (1) Can swap implementation without changing client code. (2) Methods that accept List work with any implementation. (3) Makes testing easier (mock implementations). (4) Enforces good encapsulation -- client only sees interface methods.

Q4. What are the differences between List.of(), Arrays.asList(), and new ArrayList()?

`List.of()` (Java 9+): truly immutable, null-hostile, $O(1)$ or $O(n)$ depending on size. `Arrays.asList()`: fixed-size view of an array -- `set()` allowed but `add()/remove()` throw UOE; null allowed; reflects changes to original array. `new ArrayList<>()`: fully mutable; null allowed; completely independent.

Part 2 ArrayList

Q5. How does ArrayList grow when capacity is exceeded?

`ArrayList` allocates a new array at approximately 1.5x the old size: `newCapacity = oldCapacity + (oldCapacity >> 1)`. All elements are copied to the new array. This is why appending is $O(1)$ amortized: while individual resizes are $O(n)$, they happen exponentially less frequently.

Q6. When should you specify initialCapacity for ArrayList?

When you know in advance approximately how many elements will be added. Avoids repeated `resize+copy` cycles. Rule of thumb: if you will add N elements, initialise with N or $N*1.2$ to avoid the last `resize`. `new ArrayList<>(1_000_000)` pre-allocates the array without any elements.

Q7. What is the complexity of ArrayList.contains()? Can it be improved?

$O(n)$ -- linear scan comparing each element with `equals()`. To improve to $O(1)$: use a `HashSet` for membership testing alongside the `ArrayList`. If you need both order and $O(1)$ lookup, maintain both: `ArrayList` for order, `HashSet` for `contains()` checks.

Q8. Why is ArrayList generally faster than LinkedList for iteration despite both being O(n)?

CPU cache locality. `ArrayList` stores elements contiguously in memory -- sequential access benefits from CPU prefetching and cache lines (typically 64 bytes = 16 ints fetched at once). `LinkedList` nodes are scattered in heap memory -- each `next()` follows a pointer to a random memory location, causing frequent cache misses. Benchmark studies show `ArrayList` iterates 2-5x faster in practice.

Part 3 LinkedList & CopyOnWriteArrayList

Q9. When is LinkedList genuinely better than ArrayList?

Genuinely better only for: (1) Frequent $O(1)$ insertions/deletions at HEAD (prepend). (2) Implementing a queue/deque where head and tail operations dominate. However, `ArrayDeque` is usually a better Deque implementation than `LinkedList` because of better cache locality. `LinkedList`'s middle insert/remove is $O(n)$ same as `ArrayList`.

Q10. Explain CopyOnWriteArrayList's snapshot iterator semantics.

The iterator is created with a reference to the current backing array (the snapshot). Subsequent writes to the list create a NEW array -- the iterator still holds the old reference. Therefore: iterator sees elements as they were at creation time, cannot see new elements added after creation, and `iterator.remove()` throws `UnsupportedOperationException`.

Q11. Why is `CopyOnWriteArrayList` unsuitable for high-write workloads?

Every structural modification (add, remove, set) creates a full copy of the backing array: $O(n)$ time and $O(n)$ space. Under high write throughput, this generates enormous GC pressure from short-lived array copies and burns CPU on array copying. Better alternatives for high-concurrency write scenarios: `ConcurrentLinkedQueue`, `ConcurrentHashMap`-based sets, or lock-based approaches with fine-grained locking.

Part 4 Iteration & `ConcurrentModificationException`

Q12. What causes `ConcurrentModificationException` and how does it work internally?

`ArrayList` maintains a `modCount` field incremented on every structural modification. When an `Iterator` is created, it captures `expectedModCount = modCount`. On every `it.next()` call, it checks if `(modCount != expectedModCount)` -- if not equal, throws CME. This is a best-effort check, not a guarantee (race conditions in multi-threaded code may miss it).

Q13. What are the four safe ways to remove elements during iteration?

(1) `Iterator.remove()` -- removes the last element returned by `next()`. (2) `list.removeIf(predicate)` -- Java 8+, cleanest approach. (3) Collect elements to remove, then call `list.removeAll()`. (4) Iterate backward by index: `for (int i=list.size()-1; i>=0; i--) if (condition) list.remove(i)`.

Q14. What is the difference between `Iterator` and `ListIterator`?

`Iterator`: forward-only, has `hasNext()/next()/remove()`. Works on any `Iterable`. `ListIterator`: bidirectional (`hasPrevious()/previous()`), supports `add(E)` and `set(E)` during iteration, returns index (`nextIndex()`, `previousIndex()`). Only works on `List` implementations.

Q15. Is `ConcurrentModificationException` guaranteed to be thrown for all concurrent modifications?

No -- the Javadoc says it is 'best effort'. In multi-threaded code without synchronisation, the JIT compiler or CPU may reorder operations, meaning the `modCount` check might not always see the latest value. CME is reliable only for single-threaded structural modification during iteration. For multi-threaded iteration, use `CopyOnWriteArrayList` or synchronize externally.

Part 5 Immutable Lists

Q16. What is the difference between `Collections.unmodifiableList()` and `List.of()`?

`Collections.unmodifiableList()` returns a `VIEW` over a mutable list -- the underlying list can still be modified by whoever holds a reference to it, and callers will see the change through the view. `List.of()` creates a truly immutable list -- no modifications possible at all (no add, set, remove, clear). `List.of()` is also null-hostile (throws `NPE` for null elements).

Q17. What does `Arrays.asList()` return and what are its quirks?

A fixed-size `List` backed by the array. Quirks: (1) `set()` is allowed -- modifies the backing array. (2) `add()` and `remove()` throw `UnsupportedOperationException`. (3) Changes to the original array are reflected. (4) Null elements are allowed. (5) Common mistake: `Arrays.asList(1,2,3)` returns `List`, not `List`.

Part 6 Advanced Questions [ADV]

Q18. [ADV] How does `ArrayList` handle `System.arraycopy` internally during grow and remove?

Grow: new array allocated, `System.arraycopy(old, 0, new, 0, size)`. remove(index): `System.arraycopy(elementData, index+1, elementData, index, size-index-1)`, then `elementData[--size] = null` (null out the last slot to help GC). `System.arraycopy` is a JVM intrinsic -- the JIT replaces it with a SIMD memory copy instruction, making it significantly faster than a Java loop.

Q19. [ADV] What is the `modCount` used for beyond CME detection?

`modCount` is used by the `SplitIterator` (for parallel streams) to detect concurrent structural modifications. It is also used by `subList()` views -- holding a `subList` while modifying the parent list triggers CME when you subsequently use the `subList`. Additionally, fail-fast semantics rely on `modCount` for JDK debugging tools.

Q20. [ADV] How would you implement a thread-safe `List` with better write performance than `CopyOnWriteArrayList`?

Options: (1) `ReadWriteLock` (`ReentrantReadWriteLock`) -- multiple concurrent readers, exclusive writer. Better than COWAL for large lists with frequent writes. (2) Striped locking -- partition the list into N segments, each with its own lock. (3) For append-only workloads: a lock-free `ConcurrentLinkedQueue` is efficient. (4) For MPSC: `ArrayBlockingQueue` or

LinkedTransferQueue.

Q21. [ADV] Explain the internal structure of List.of() for different sizes.

Java 9 uses different internal classes based on size for space efficiency: List12 for 0-2 elements (fields, no array), ListN for 3+ elements (array). All are immutable, null-hostile, and serialization-safe. This specialisation avoids array allocation overhead for small collections.

Q22. [ADV] How does Collections.sort() work internally in Java 8+?

List.sort() (called by Collections.sort()) uses TimSort: a hybrid merge sort + insertion sort. It detects already-sorted runs (natural runs) in the data and merges them efficiently. Best case $O(n)$ for already sorted. Average/worst $O(n \log n)$. Stable: equal elements maintain original order. Requires array-backed list -- LinkedList performance degrades because get(i) is $O(n)$.

Q23. [ADV] What happens when you call subList() and then add to the parent list?

Structural modification (add/remove) to the parent list after creating a subList invalidates the subList. Subsequent use of the subList (including iteration, get, size) throws ConcurrentModificationException because the subList captures the parent's modCount at creation. Fix: use the subList before modifying the parent, or make an independent copy: new ArrayList<>(sub).

Q24. [ADV] How does ArrayList implement Serializable and what is elementData transient?

elementData is marked transient to exclude it from default Java serialization. ArrayList provides custom writeObject/readObject methods that serialize only the size and the actual elements (indices 0 to size-1), not the full capacity. This prevents serializing unused null slots at the end of the array, reducing serialized size.

Q25. [ADV] What is the difference between removeIf() and stream().filter().collect() for removing elements?

removeIf() mutates the original list in-place using a single pass ($O(n)$, no allocation). stream().filter().collect() creates a new list with passing elements ($O(n)$ time + $O(n)$ space for new list). removeIf() is more memory-efficient when you want to mutate. stream().filter().collect() is better when you want to preserve the original.

Part 7 Real-World Scenarios [ADV]

Q26. [ADV] A service adds elements to a shared List from multiple threads. What happens and how do you fix it?

Without synchronisation: data corruption (two threads write to the same index), size counter inconsistency, and ConcurrentModificationException during iteration. Fix options: (1) If writes are rare and reads dominate: CopyOnWriteArrayList. (2) If concurrent writes are frequent: Collections.synchronizedList(new ArrayList<>()) -- but iterators must be externally synchronized. (3) For producer-consumer: BlockingQueue (LinkedBlockingQueue, ArrayBlockingQueue).

Q27. [ADV] How would you efficiently remove all occurrences of a value from a large ArrayList?

Option 1: list.removeIf(x -> x.equals(value)) -- $O(n)$, single pass, in-place. Internally uses a compaction algorithm: tracks a 'write' index and copies elements that don't match. Option 2: loop backward with remove(index) -- $O(n^2)$ for many matches (each remove shifts elements). Option 3: collect non-matching elements into new list -- $O(n)$ but allocates. removeIf is always the best choice for this pattern.

Q28. [ADV] What is the time complexity of a complete sort of a List and what factors affect it?

Collections.sort() / List.sort() uses TimSort: $O(n \log n)$ average and worst case. Factors: (1) Already sorted or nearly sorted -> $O(n)$ (TimSort optimisation). (2) Comparator complexity: each comparison call cost adds a constant factor. (3) Autoboxing: List.sort() with comparator boxing is slower than int[] Arrays.sort(). (4) List type: LinkedList sort is slower due to $O(n)$ get(i); ArrayList benefits from array locality.

EX Exercises, Debug Challenges & MCQ

Part A Coding Challenges

- A1.** Implement a method `List<> partition(List list, int size)` that splits a list into sublists of exactly size elements (last sublist may be smaller). Do not modify the input list.
- A2.** Write a method that removes duplicates from an `ArrayList` while preserving insertion order. Solve it in $O(n)$ time.
- A3.** Implement a circular buffer (ring buffer) backed by an `ArrayList` with a fixed capacity. Support `add(E)`, `poll()`, and `peek()` in $O(1)$.
- A4.** Write a method `List flattenDeep(List nested)` that flattens arbitrarily deeply nested List structures into a single List using recursion.

Part B Debug & Fix

```
// B1 ConcurrentModificationException

List<String> list = new ArrayList<>(List.of("a","b","c"));

for (String s : list) {

    if (s.equals("b")) list.remove(s); // CME!

}

// FIX: list.removeIf(s -> s.equals("b"));

// OR: use Iterator.remove()
```

B1: Fix ConcurrentModificationException

```
// B2 subList mutation trap

List<Integer> nums = new ArrayList<>(List.of(1,2,3,4,5));

List<Integer> sub = nums.subList(1,4); // [2,3,4]

nums.add(6); // structural change!

System.out.println(sub.size()); // ConcurrentModificationException!

// FIX: List<Integer> sub = new ArrayList<>(nums.subList(1,4)); (independent copy)
```

B2: Fix subList invalidation after parent modification

```
import java.util.Arrays;
import java.util.List;
import java.util.ArrayList;

// B3 Arrays.asList() fixed-size trap

List<String> list = Arrays.asList("a","b","c");

list.add("d"); // UnsupportedOperationException!

// FIX: List<String> list = new ArrayList<>(Arrays.asList("a","b","c"));

// OR: List<String> list = new ArrayList<>(List.of("a","b","c"));
```

B3: Fix Arrays.asList() fixed-size trap

Part C MCQ: Predict the Output

#	Code	Answer	Why
M C1	List<String> l=List.of("a","b"); l.add("c");	UOE	List.of() is immutable
M C2	List<String> l=Arrays.asList("a","b"); l.set(0,"x"); println(l);	[x,b]	set() allowed on Arrays.asList()
M C3	List<String> l=new ArrayList<>(); l.add(null); println(l.size());	1	ArrayList allows null
M C4	List.of(null)	NPE	List.of() is null-hostile
M C5	new ArrayList<>(List.of("a","b")).removeIf(s->s.equals("a")); size?	1	removeIf removes matching

REF Quick Reference Cheat Sheet

Topic	Key Points to Remember
Collection hierarchy	Iterable->Collection->List/Set/Queue. Map is SEPARATE (not-a-Collection).
ArrayList growth	newCapacity = old + (old>>1) approx 1.5x. Pre-size with initialCapacity for bulk adds.
ArrayList complexity	get/set= $O(1)$. add(end)= $O(1)$ amortized. add/remove(middle)= $O(n)$. contains= $O(n)$.
LinkedList use case	$O(1)$ head/tail ops. But ArrayDeque is usually faster for deque ops. Poor cache locality.
COWAL semantics	Writes: $O(n)$ copy. Reads: lock-free on snapshot. Iterator: snapshot, remove() throws UOE.
CME cause	modCount check: structural modification during iteration (not remove()).
CME fixes	Iterator.remove() / removeIf() / removeAll() / backward index loop.
Fail-fast vs safe	Fail-fast: throws CME. Fail-safe (COWAL): iterates snapshot, no CME.
List.of()	Truly immutable. Null-hostile. Java 9+. May use specialised compact impl.
unmodifiableList()	VIEW -- underlying list can still change through original reference!
Arrays.asList()	Fixed size. set() OK, add/remove throw UOE. Reflects array changes.
Stream.toList()	Java 16+. Returns unmodifiable list. Cleaner than collect(Collectors.toList()).
subList()	Returns a LIVE VIEW. Structural mod to parent -> CME on subList. Copy if needed.
Sort stability	List.sort() uses TimSort -- stable. Equal elements maintain relative order.
Thread safety	ArrayList not safe. Use COWAL (read-heavy), synchronizedList (write OK with external sync).